Security Review Report
NM-0607 Defx Bridge

NETHERMIND
SECURITY

(August 27, 2025)

Contents

1 Executive Summary

This document presents the results of the security review conducted by Nethermind Security for Defx bridge contracts. The `DefxBridge` contract facilitates the transfer of ERC20 tokens and native assets across multiple networks. Its operations are governed by a dynamic validator set, which employs a dual-key authorization system (hot and cold wallets).

The audit focused on the upcoming upgrade to the `DefxBridge` contract. The upgrade introduces the following changes:

- Support for deposits and withdrawals of native tokens.
- Transfer of upgrade authority from a single contract owner to a validator quorum.
- Revised withdrawal structure that allows large transfers to be executed in a single request.

The audit comprises 1072 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report four points of attention, where two are classified as `Low` and two are classified as `Informational` or `Best Practices` severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 concludes the document.

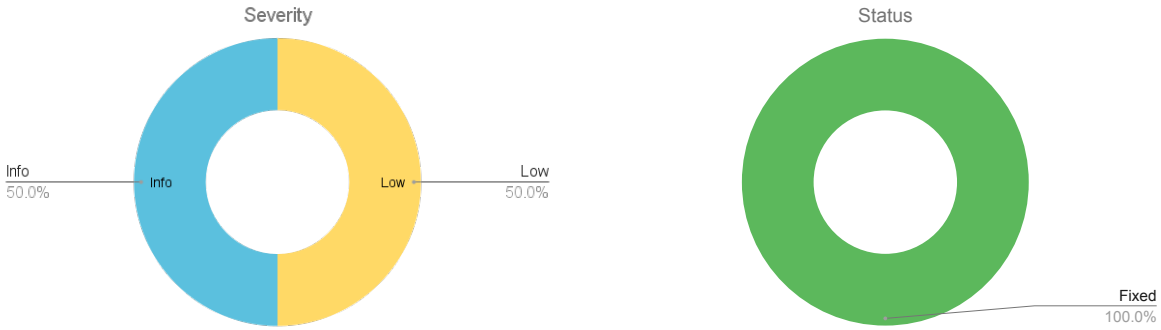

Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (0), **Low** (2), **Undetermined** (0), **Informational** (2), **Best Practices** (0). **Distribution of status: Fixed** (4), **Acknowledged** (0), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	August 22, 2025
Final Report	August 27, 2025
Initial commit	f2d48d801c8cd9f6788874912ab47d2c993a84f6
Final commit	ee8c0164ca52e62e6079e32f5d1fe21e232968f1
Documentation Assessment	Low
Test Suite Assessment	No tests were provided

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	DefxBridge.sol	744	147	19.8%	122	1013
2	library/SignatureLibrary.sol	53	1	1.9%	9	63
3	library/ValidatorLibrary.sol	126	3	2.4%	18	147
4	common/Events.sol	46	1	2.2%	18	65
5	common/Structs.sol	67	3	4.5%	10	80
6	common/Errors.sol	36	1	2.8%	12	49
	Total	**1072**	**156**	**14.6%**	**189**	**1417**

3 Summary of Issues

	Finding	Severity	Status
1	Old signatures can be reused to create withdrawal requests	Low	Fixed
2	Signature version is not updated in the upgraded contract	Low	Fixed
3	CEI pattern not respected in `finalizeWithdrawal`	Info	Fixed
4	Deposit amounts are limited to `uint64`	Info	Fixed

4 System Overview

The Defx bridge allows for the transfer of assets between EVM-compatible blockchains. It initially supports Arbitrum and Defx's proprietary Layer 1 (L1) blockchain. This bridge is governed by a group of validators responsible for deposits, withdrawals, and overall bridge management.

This audit reviewed the recent upgrade to the bridge contract, which introduces the following changes:

- **Support for native tokens**: Previously, the bridge supported only ERC20 tokens. The upgrade extends functionality to include deposits and withdrawals of native tokens. Validators can collectively turn this feature on or off through quorum-based governance by toggling the newly introduced `isNativeTokenEnabled` flag.
- **Transfer of upgrade authority**: The `DefxBridge` follows the UUPS upgradeable pattern. In earlier versions, only the contract owner held upgrade authority. Under the new design, the validator set collectively approves the new implementation through quorum voting. Once an implementation has been approved, the upgrade transaction can be executed by any party.
- **Withdrawal amount limit**: The withdrawal mechanism was previously limited by the use of a `uint64` data type. To support significantly larger withdrawals, this has been updated to `uint256`. To preserve storage layout compatibility and avoid disrupting existing state, the legacy `requestedWithdrawals` mapping has been deprecated, and a new mapping, `requestedWithdrawalsV2`, is introduced to handle withdrawal requests post-upgrade.
- **Quorum majority criteria**: In prior versions, proposals required strictly more than two-thirds of validator votes to pass. This rule has been revised to approve proposals when validator votes are greater than or equal to two-thirds.

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Old signatures can be reused to create withdrawal requests

File(s): DefxBridge.sol

Description: During the recent upgrade, the contract introduced a new mapping `requestedWithdrawalsV2` to replace the legacy `requestedWithdrawals`. This mapping stores withdrawal requests keyed by their message hash and maps them to a `WithdrawalData` structure. The primary change in `WithdrawalData` is that the `amount` field type was updated from `uint64` to `uint256`, while all other fields remain unchanged.

In the original implementation, replay protection was enforced by checking whether the generated message hash already existed in the `requestedWithdrawals` mapping. If a user attempted to resubmit the same signatures, the same hash would be regenerated and the transaction would revert, effectively preventing signature reuse.

However, after the upgrade, replay protection is only applied against the newly initialized `requestedWithdrawalsV2` mapping, leaving the populated `requestedWithdrawals` mapping ignored. As a result, old valid signatures can be reused to submit withdrawal requests, which will be stored in the new mapping and emit `RequestedWithdrawal` events. However, these withdrawals cannot be finalized because the `finalized` mapping is checked during finalization, preventing any actual double withdrawals from occurring.

```
function batchRequestWithdrawals(
    RequestWithdrawalV2[] calldata withdrawals
) external whenNotPaused nonReentrant {
    // ...
    for (uint256 i = 0; i < withdrawals.length; i++) {
        bytes32 messageHash = SignatureLibrary.generateUniqueMessageHash(
            keccak256(
                abi.encode(
                    "batchRequestWithdrawals",
                    withdrawals[i].user,
                    withdrawals[i].amount,
                    withdrawals[i].token,
                    withdrawals[i].nonce
                )
            ),
            address(this)
        );
        // ...
        // Check if the withdrawal has been requested
        if (
            requestedWithdrawalsV2[messageHash]
                .requestedEpochTimestampInSeconds != 0
        ) {
            emit WithdrawalFailed(messageHash, 2);
            return;
        }

        // ...
        requestedWithdrawalsV2[messageHash] = withdrawalData;
        emit RequestedWithdrawal(withdrawalData);
    }
}
```

Recommendation(s): Extend replay protection to check both `requestedWithdrawals` and `requestedWithdrawalsV2` mappings to prevent re-requesting old withdrawals. Additionally, implement a check for finalized withdrawals to ensure they cannot be requested in any future contract state.

Status: Fixed

Update from the client: Changes made:

- Added replay protection to check the old `requestedWithdrawals` mapping;
- Added check for `finalizedWithdrawals` mapping to prevent reuse of already finalized withdrawal signatures;
- Now the function checks all three mappings: `requestedWithdrawals`, `requestedWithdrawalsV2`, and `finalizedWithdrawals`;

6.2 [Low] Signature version is not updated in the upgraded contract

File(s): SignatureLibrary.sol

Description: The DefxBridge contract implements EIP-712 domain separation for structured signature validation. As part of this scheme, the version field plays a critical role in distinguishing signatures across different contract implementations and upgrades. In the upgraded implementation of DefxBridge, however, the version remains hardcoded to "1":

```
function makeDomainSeparator() internal view returns (bytes32) {
        return
            keccak256(
                abi.encode(
                    EIP712_DOMAIN_SEPARATOR,
                    keccak256(bytes("DefxBridge")),
                    //@audit version is not updated
                    keccak256(bytes("1")),
                    block.chainid,
                    address(this)
                )
            );
    }
```

Since the domain separator was not updated to reflect the new contract version, signatures created under the previous implementation are still considered valid by the upgraded contract. This defeats the purpose of versioning and prevents the system from cryptographically separating signatures between different contract iterations. While this design flaw on its own is not an immediate critical exploit, it weakens the replay protection mechanism.

Recommendation(s): Update the version field in the domain separator (e.g., 2) during this contract upgrade to ensure signatures are unique to the current implementation.

Status: Fixed

Update from the client: Changes made:

- Updated the version from "1" to "2" in the EIP-712 domain separator;
- This ensures signatures from the previous contract version cannot be replayed in the upgraded contract;

6.3 [Info] CEI pattern not respected in `finalizeWithdrawal`

File(s): DefxBridge.sol

Description: The `finalizeWithdrawal` function performs a low-level call to the user's address when handling native token payments. However, the state variable `finalizedWithdrawals` is updated after this external call, violating the Checks-Effects-Interactions (CEI) pattern.

```
function finalizeWithdrawal(...) external whenNotPaused nonReentrant {
    // ...
    // Finalize the withdrawals
    WithdrawalDataV2 memory withdrawal = requestedWithdrawalsV2[message];
    if (withdrawal.token == address(0)) {
        // ...
        //@audit External call to user
        (bool success, ) = withdrawal.user.call{value: withdrawal.amount}(
            ""
        );
        if (!success) {
            emit WithdrawalFailed(message, 7);
            return;
        }
    } else {...}
    // @audit state variable updated after the external call
    finalizedWithdrawals[message] = true;
    emit FinalizedWithdrawal(requestedWithdrawalsV2[message]);
}
```

Although most functions are currently permissioned and protected with the `nonReentrant` modifier, this design could expose the contract to potential reentrancy risks if a user-facing function is introduced in the future.

Recommendation(s): Update the contract's state (`finalizedWithdrawals`) before executing the external call to ensure compliance with the CEI pattern and reduce reentrancy risk. In the case where the external call fails (`!success`), revert the state update.

Status: Fixed

Update from the client: Changes made:

- Moved the state update (`finalizedWithdrawals[message] = true`) before external calls;
- Added proper state reversion logic in case of failed external calls;
- Now follows the **Checks-Effects-Interactions (CEI)** pattern correctly;

6.4 [Info] Deposit amounts are limited to `uint64`

File(s): DefxBridge.sol

Description: In the current upgrade, the withdrawal mechanism was updated to use `uint256` for the withdrawal amount, removing the previous limitation imposed by `uint64`. This change allows users to perform larger withdrawals without being constrained by the smaller data type.

However, the same limitation remains for deposits with permit. The `DepositWithPermit` struct continues to define the `amount` field as a `uint64`, which restricts the maximum deposit size. This inconsistency between deposits and withdrawals may cause unnecessary constraints for users.

```
struct DepositWithPermit {
    address user;
    //@audit deposits are limited to uint64
    uint64 amount;
    uint64 deadline;
    address token;
    Signature signature;
}
```

Recommendation(s): Review whether deposit amounts are intended to be limited. If not, update the `amount` field in the `DepositWithPermit` struct to `uint256` to align with withdrawals and avoid deposit size restrictions.

Status: Fixed

Update from the client: Changes made:

- Updated the `amount` field from `uint64` to `uint256`;
- This aligns deposit functionality with withdrawal functionality and removes 64-bit integer ceiling;

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Defx's documentation

The `Defx` team provided a clear presentation of the bridge changes during the kick-off call and effectively addressed the questions the Nethermind Security team raised throughout subsequent discussions. Nonetheless, the documentation could be strengthened by adding more NatSpec comments directly in the codebase and providing written documentation outlining the bridge logic and core functionalities.

8 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.